idm

Downloader with Internet Download Manager (Windows), with an HTTP **agent server**, a **web interface** and **download completion tracking**. Links can be sent to IDM from any OS.

- Features
- Installing
- Example
- Command line usage
- Agent server mode
  - Architecture
  - Quick start
  - Web interface
  - Download completion tracking
  - HTTP API
  - Job lifecycle
  - FAQ: what is the queue for?
  - Python API
  - Configuration
  - Security
  - Run at Windows startup
  - Troubleshooting
- Support
- Tips
- Links
- Author

Features

- Send links to IDM from Python or the command line (referrer, cookies, post data, credentials, user agent, save path, add-only).
- **Agent server** (`idm --server`): receive links over HTTP/JSON and forward them to IDM through COM.
- **Agent client** (`idm -A`): send links to an agent from Windows, Linux, macOS or anywhere Python 3.7+ runs.
- **Web interface** served by the agent at `/`: add links, watch every IDM download.
- **Completion tracking**: knows if a download is still running, complete, failed or cancelled (reads IDM's own log files). `idm -w` waits for it.
- Token authentication, same-origin/CSRF/DNS-rebinding protection, loopback-only by default.

- Standard library only for the server, client and web UI.

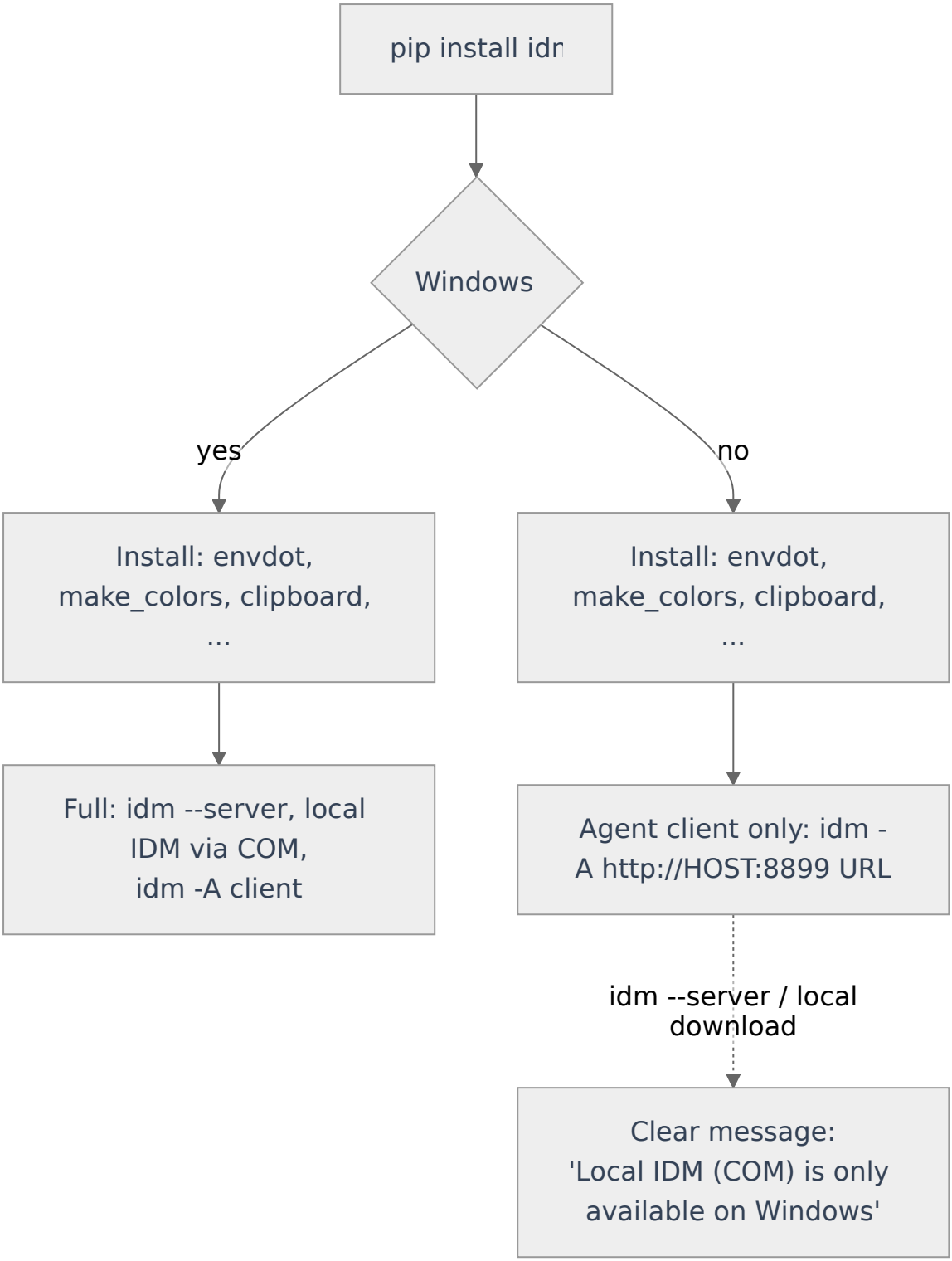
Installing

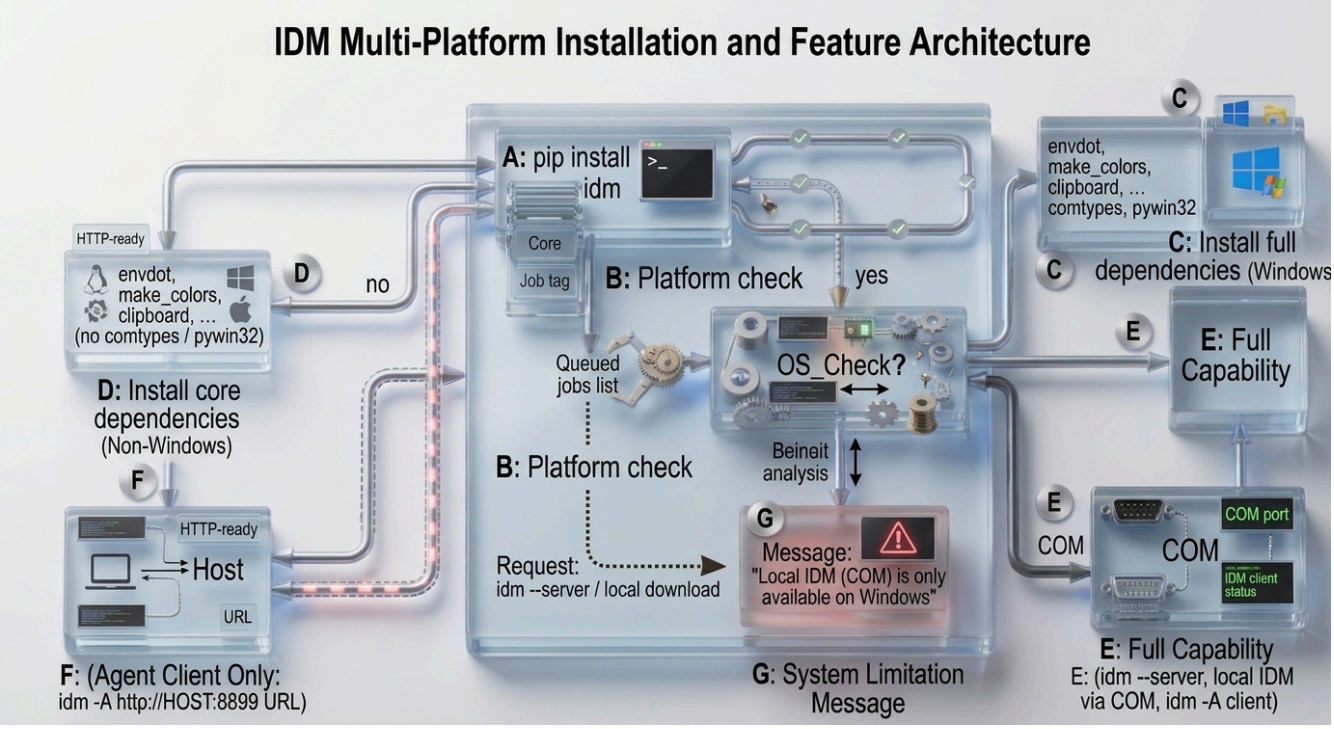
Install and update using pip:

```
pip install idm
```

Requires Python 3.7+. There are two install types and pip picks the right one for the OS:

	Windows	Linux / macOS / other
`pip install idm` (default, everything)	agent server + COM + client; also installs `comtypes` and `pywin32`	**agent client only** (no COM packages are installed, they are Windows-only markers)
`idm-agent` (agent only, built with `PYIDM_AGENT_ONLY=1`)	client only, never installs `comtypes` / `pywin32`	client only


pip install idm
Windows
yes
no
Install: envdot, make_colors, clipboard, ...
Install: envdot, make_colors, clipboard, ...
Full: idm --server, local IDM via COM, idm -A client
Agent client only: idm -A http://HOST:8899 URL
idm --server / local download
Clear message: 'Local IDM (COM) is only available on Windows'



On non-Windows systems `import idm` works and `idm -A ...` works; `idm --server` and local downloads print a clear message and exit with code 1 instead of crashing.

Agent-only build (for example a Windows PC that only forwards links to another PC):

```
set PYIDM_AGENT_ONLY=1        # Linux/macOS: export PYIDM_AGENT_ONLY=1
pip wheel . --no-deps -w dist # builds idm_agent-*.whl, then: pip install dist/idm_agent-*.whl
```

`idm` and `idm-agent` install the same `idm` package, install only one of them. On a Windows machine that was installed agent-only you can add the COM packages later with `pip install comtypes pywin32`.

Example

What does it look like? Here is an example of a simple pyidm program:

```
from idm import IDMan

downloader = IDMan()
url = "http://test.com/test.exe"

downloader.download(url, r"c:\DOWNLOADS", "test_output.exe")
```

and "Internet Download Manager (IDM)" will open then immediately download the URL

add link without start downloading

```python
from idm import IDMan

downloader = IDMan()
url = "http://test.com/test.exe"

downloader.download(url, r"c:\DOWNLOADS", "test_output.exe", add_only=True)
```

`download()` returns `True` when IDM accepted the link, `False` otherwise (the reason is in `downloader.last_error`). `IDMan()` itself never fails; `download()` raises `OSNotSupport` (not Windows / `comtypes` missing) or `IDMNotFound` (IDM not installed).

or run on terminal/cmd

```
> idm "http://test.com/test.exe" -p C:\DOWNLOADS -o test_output.exe -c -ua "Mozilla/5.0 (Windows
# or pyidm
```

Example use with headers as postData, cookies and other parameters:

```python
from idm import IDMan

downloader = IDMan()
url = "http://test.com/test.exe"

headers = {
    'Authorization': "Bearer KN9WW7k5gKgCnQLpnmWWM7LytAuSUwb9",
    'Cookie': 'accountToken=KN9WW7k5gKgCnQLpnmWWM7LytAuSUwb9',
}
cookies = {
    'accountToken': 'KN9WW7k5gKgCnQLpnmWWM7LytAuSUwb9'
}

username = "admin"
password = "admin123"
confirm = True
user_agent = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Ch

downloader.download(url, r"c:\DOWNLOADS", "test_output.exe", "http://test.com/test.exe", cookies
```

Command line usage

you can use `idm` or `pyidm`

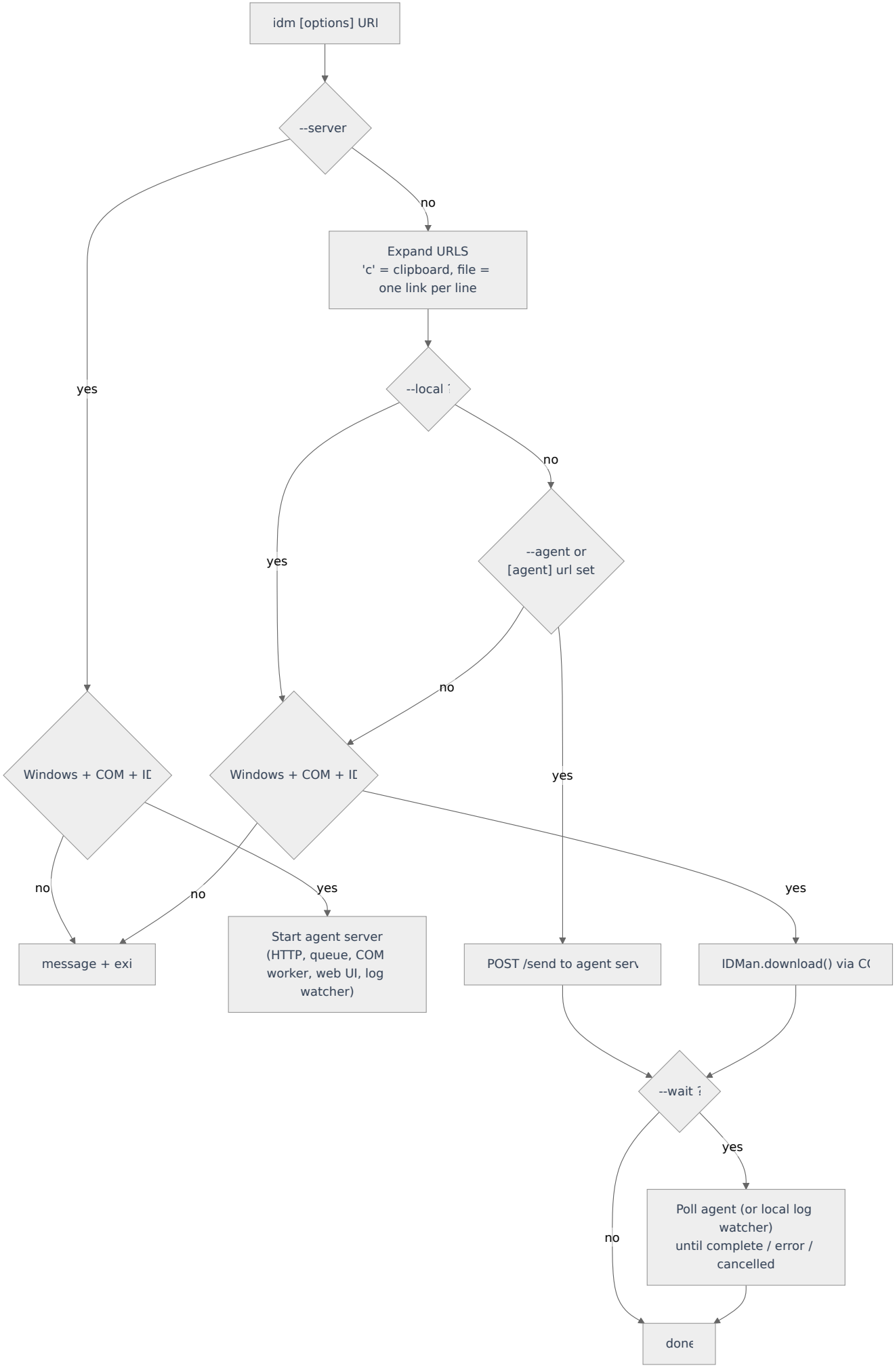
```
usage: idm [-h] [-p PATH] [-o OUTPUT] [-c] [-a] [-r REFERRER] [-C COOKIE] [-D POST_DATA] [-U USER
           [--agent-token AGENT_TOKEN] [--local] [-w] [--timeout TIMEOUT] [--no-webui] [--no-wato
           [URLS ...]
```

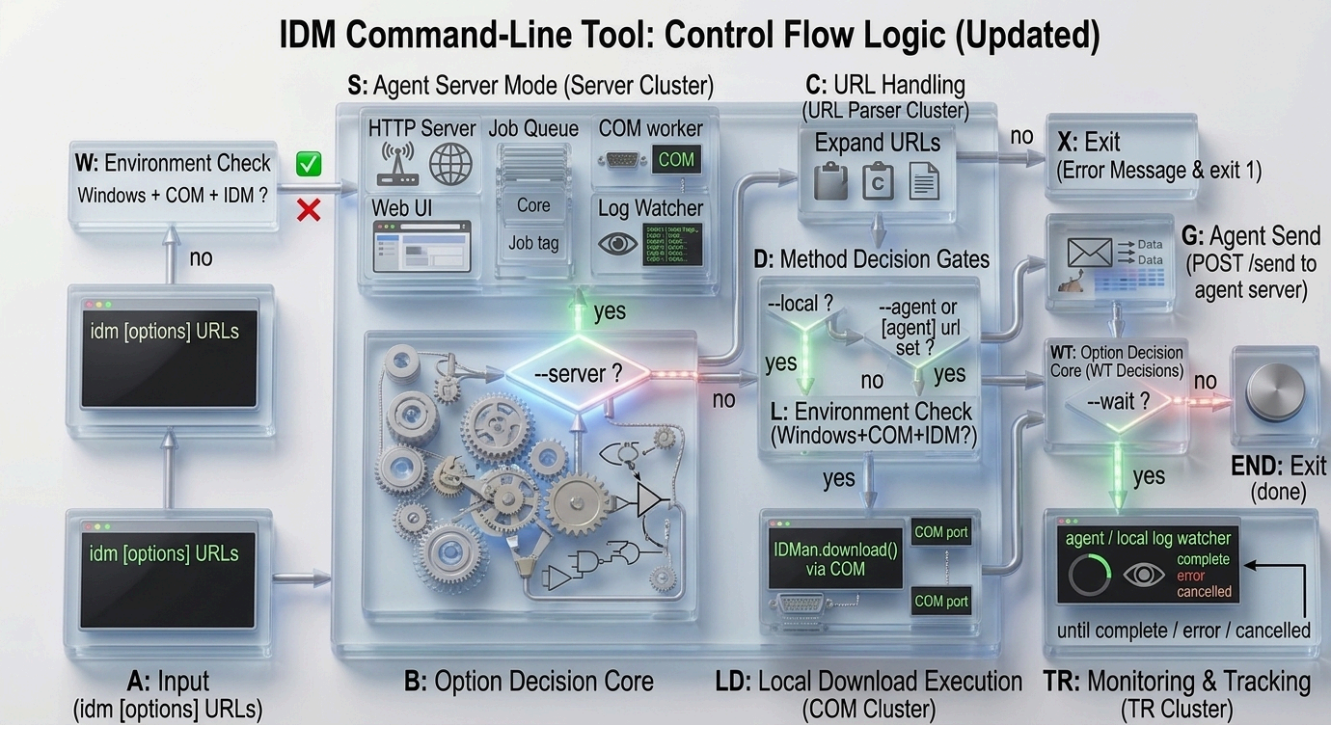
```
Command line downloader with/Via Internet Download Manager(IDM)

Positional Arguments:
  URLS                  url to download, or "c" to get url from clipboard or a text file contain

Options:
  -h, --help            show this help message and exit
  -p, --path PATH       Path to save (default: current dir; on agent server: its configured path)
  -o, --output OUTPUT   Save with different name
  -c, --confirm         Confirm before download
  -a, --add             Add link to IDM without start downloading
  -r, --referrer REFERRER
                        Url referrer
  -C, --cookie COOKIE   Cookie string or dict
  -D, --post-data POST_DATA
                        Post Data string or dict
  -U, --username USERNAME
                        Username if require
  -P, --password PASSWORD
                        Password if require
  -ua, --user-agent USER_AGENT
                        Send with custom User-Agent string
  -s, --server          Run agent server: receive links over HTTP and forward them to IDM (COM)
  --host HOST           Agent server bind host (default: 127.0.0.1)
  --port PORT           Agent server port (default: 8899)
  --token TOKEN         Agent server token (or env PYIDM_TOKEN), required if host is not loopback
  -A, --agent AGENT     Send links to an agent server instead of local IDM, e.g. http://192.168.1
  --agent-token AGENT_TOKEN
                        Token for the agent server (or env PYIDM_TOKEN)
  --local               Ignore configured agent url, send to local IDM
  -w, --wait            Wait until the download is complete (needs the IDM log watcher), exit coo
  --timeout TIMEOUT     Max seconds to wait with --wait (default: 3600, 0 = forever)
  --no-webui            Agent server: disable the web interface
  --no-watcher          Agent server: disable IDM log watching (download state tracking)
  --log-dir LOG_DIR     IDM log directory (default: %APPDATA%\IDM\DwnlData\<USERNAME>)
  -v, --verbose         Verbose agent server log
  --config CONFIG       set config, format section:option:value, for list valid section/option ty
```

How the CLI decides where a link goes:

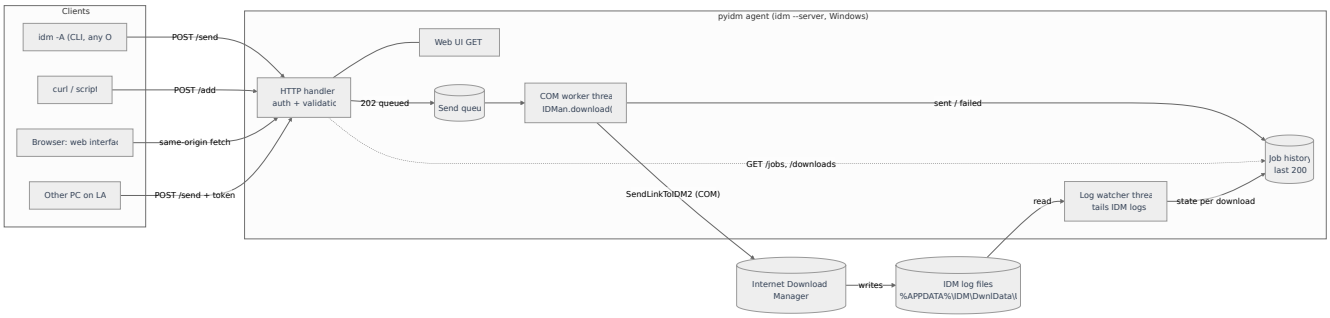

idm [options] URI
--server
yes
no
Expand URLS
'c' = clipboard, file =
one link per line
--local ?
yes
no
--agent or
[agent] url set
yes
no
Windows + COM + ID
Windows + COM + ID
yes
no
no
yes
yes
message + exi
Start agent server
(HTTP, queue, COM
worker, web UI, log
watcher)
POST /send to agent serv
IDMan.download() via C(
--wait ?
yes
no
Poll agent (or local log
watcher)
until complete / error /
cancelled
done

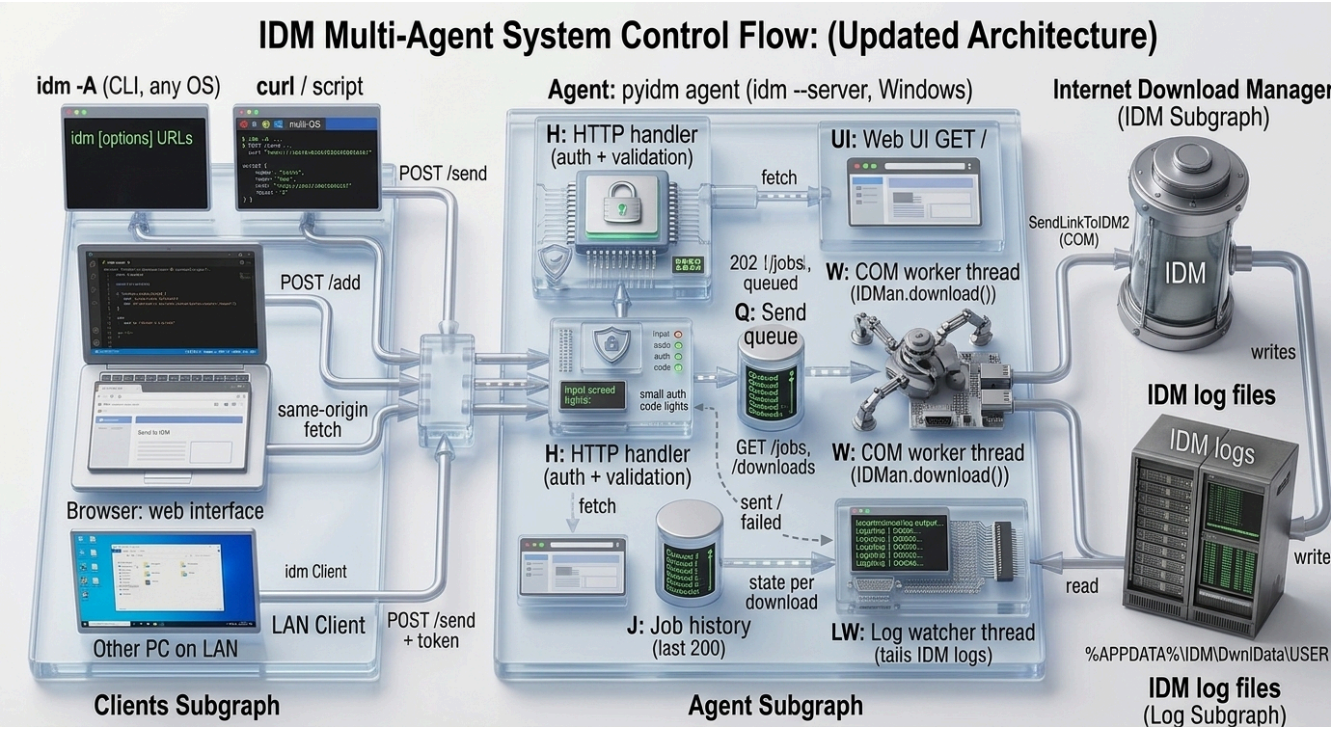


Agent server mode

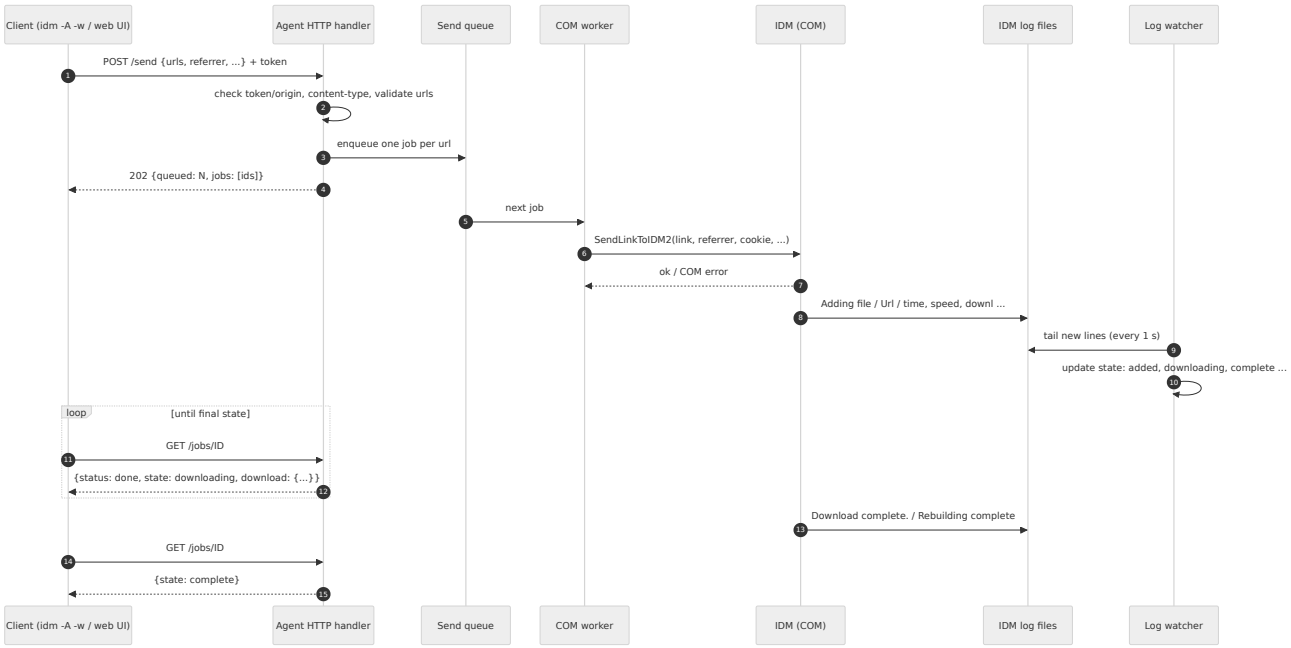
Run `idm` as a small HTTP agent on the Windows machine where IDM is installed. Any client (another PC, a script, `curl`, a browser extension, or the built-in web page) sends links to it, the agent forwards them to IDM through COM and then tells you what IDM did with them. Requests go through a queue and are executed one by one on a single COM worker thread. No extra dependency is needed (Python standard library only).

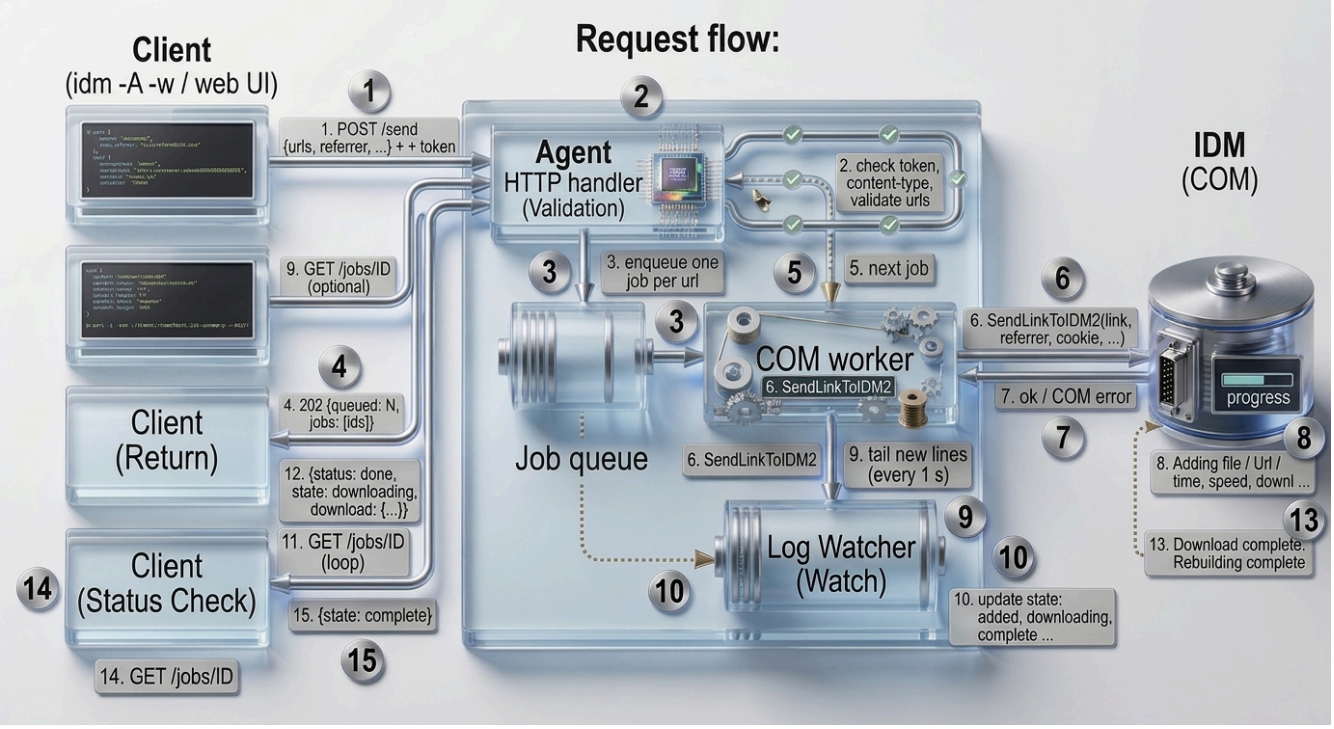
Architecture





Request flow, including completion tracking:





Quick start

Start the agent (on the Windows machine where IDM is installed):

```
> idm --server                                          # 127.0.0.1:8899, no token (loopback
> idm --server --host 0.0.0.0 --port 8899 --token SECRET    # token is REQUIRED for non-loopback
> idm --server -v                                       # log every request and job
> idm --server --no-webui --no-watcher                  # API only, no web page, no completi
```

Send links to the agent from the command line (works on any OS):

```
> idm -A http://192.168.1.10:8899 --agent-token SECRET "http://test.com/test.exe" -r "http://test
> idm -A 127.0.0.1:8899 links.txt          # text file, one link per line ('#' = comment)
> idm -A 127.0.0.1:8899 c                  # link(s) from clipboard
> idm -A 127.0.0.1:8899 -w "http://test.com/big.iso"   # wait until the download is complete (ex
```

Or with HTTP/JSON (`Content-Type: application/json` is required, token via `X-Token` or `Authorization: Bearer`):

```
curl -X POST http://127.0.0.1:8899/send -H "Content-Type: application/json" -H "X-Token: SECRET"
     -d "{\"urls\": [\"http://test.com/test.exe\"], \"referrer\": \"http://test.com\"}"
```

Web interface

Open `http://HOST:8899/` in a browser. It is a single embedded page (no external files, no CDN):

- paste one or more links, optional referrer / save path / "add only", press **Send to IDM**;
- table of the jobs sent through this agent with their live state, downloaded bytes, speed and save-as name;
- table of **all** IDM downloads found in IDM's log files (including ones you started in IDM itself);
- token field (kept in `sessionStorage`) when the agent runs with `--token`;
- refreshes every 2 seconds; disable with `--no-webui`.

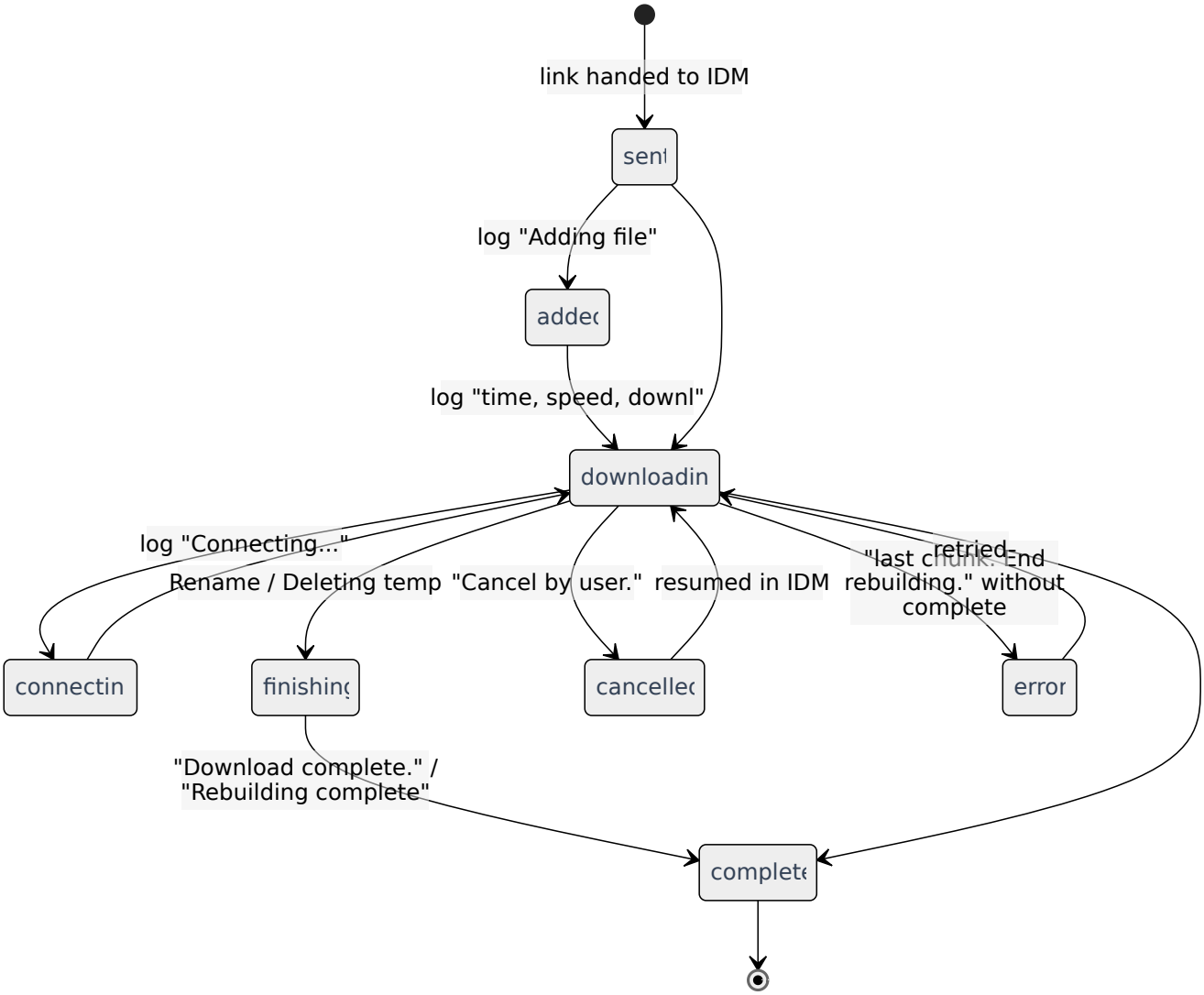
The page itself is public (it contains no data); everything it shows comes from the API, which needs the token if one is set.

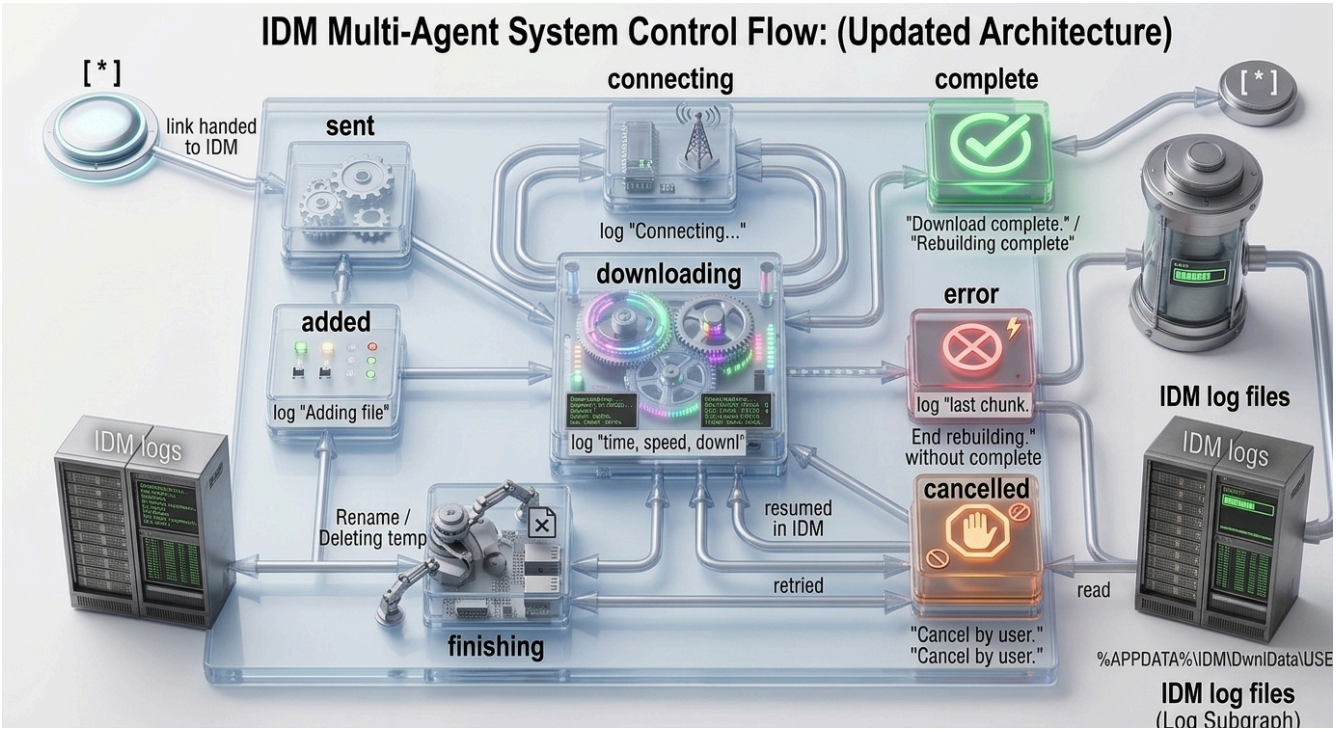
Download completion tracking

COM (`SendLinkToIDM2`) only **hands a link over** to IDM, it never reports back. So "was the link accepted" and "did the download finish" are two different questions:

Field	Answers	Source
`status`	was the link handed to IDM? `queued`, `running`, `done`, `failed`	COM call result
`state`	what is IDM doing with it? `sent`, `added`, `downloading`, `connecting`, `finishing`, `complete`, `error`, `cancelled`	IDM log files

The agent runs a small watcher thread that tails IDM's log files (`%APPDATA%\IDM\DwnlData\<USERNAME>\*.log`, override with `[watcher] log_dir` or `--log-dir`), recognises the progress lines (`Url`, `to file`, `time ..., speed ..., downl ...`, `Download complete.`, `Rebuilding complete`, `Connecting...`, `Cancel by user.`, ...) and matches each log to a job by URL. Only logs touched in the last 5 minutes are read when the agent starts, so old history does not show up.


link handed to IDM
sent
log "Adding file"
added
log "time, speed, downl"
downloadin
log "Connecting..."
Rename / Deleting temp
"Cancel by user."
resumed in IDM
"last chunk." End rebuilding." without complete
retried
connectin
finishing
cancelled
error
"Download complete." / "Rebuilding complete"
complete


IDM Multi-Agent System Control Flow: (Updated Architecture)
[*]
link handed to IDM
sent
connecting
log "Connecting..."
downloading
log "time, speed, downl"
added
log "Adding file"
Rename / Deleting temp
finishing
resumed in IDM
retried
cancelled
"Cancel by user."
"Cancel by user."
complete
"Download complete." / "Rebuilding complete"
error
log "last chunk.
End rebuilding." without complete
[*]
IDM log files
IDM logs
read
%APPDATA%\IDM\DwnlData\USE
IDM log files
(Log Subgraph)

Notes:

- `-a` / `add_only` jobs stay `added` until you start them in IDM.
- `complete` is final immediately. `error` and `cancelled` are treated as final by `--wait` only after they stayed unchanged for 10 seconds, because IDM writes some of those lines during normal retries.
- If the log directory does not exist (IDM not run yet, other user, `--no-watcher`) the agent still works; `state` stays `sent` and `/health` reports `"watcher": false`. `idm -w` then returns right after sending and says that completion cannot be tracked.
- Log line patterns come from IDM's log format as observed on current IDM versions; if a future IDM changes the wording the state stays at its last known value.

HTTP API

Endpoint	Auth	Description
`GET /` (`/ui`)	no	web interface
`POST /send`	yes	send link(s) to IDM (start download unless `add_only: true`)
`POST /add`	yes	add link(s) to IDM queue without starting (forces `add_only`)
`GET /jobs`	yes	jobs sent through this agent, newest first
`GET /jobs/<id>`	yes	one job
`GET /downloads`	yes	every download seen in IDM's log files
`GET /health`	no	`{"status":"ok","version":"...","pending":0,"watcher":true}`

Request body (JSON object):

Field	Type	Description
`url` / `urls`	string / list	link(s), only `http`, `https`, `ftp` (max 500 per request)
`path`	string	save directory on the **agent** machine (default: `[download] path` of the agent)
`output`	string	save with different name
`referrer`	string	url referrer

Field	Type	Description
`cookie`	string / object	cookie string or `{name: value}`
`post_data`	string / object	post data / extra headers
`username`, `password`	string	credentials if required
`confirm`	bool	confirm dialog in IDM before download
`user_agent`	string	custom User-Agent
`add_only`	bool	add to IDM queue without starting

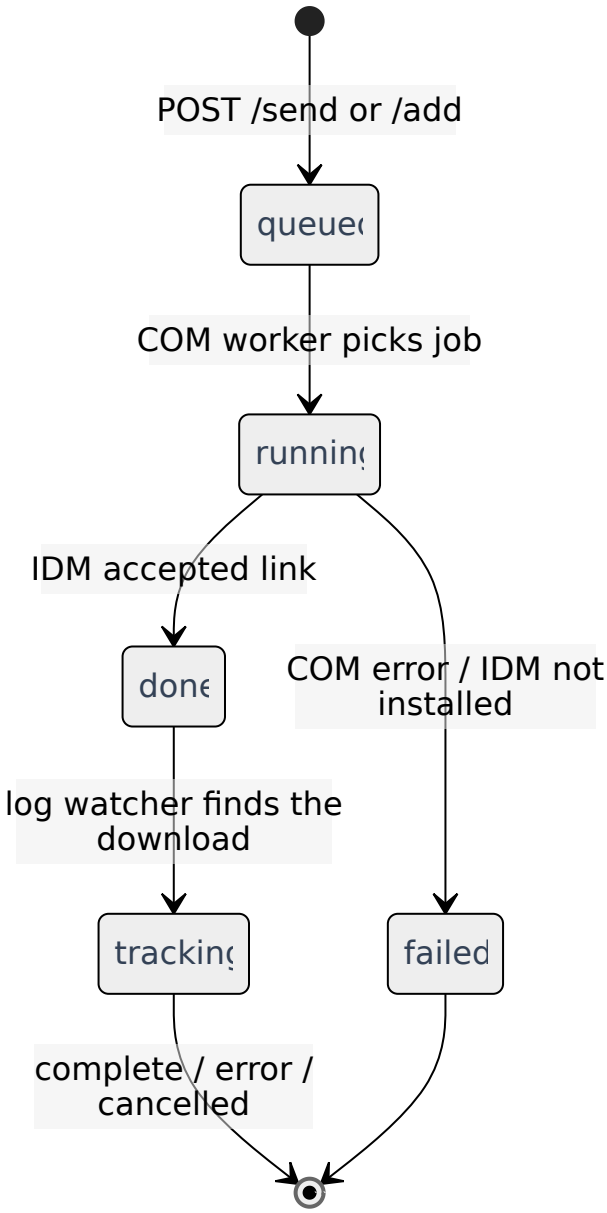
Responses:


```
{"queued": 2, "jobs": ["a1a8d016346c", "3701fe02acc4"]}
```

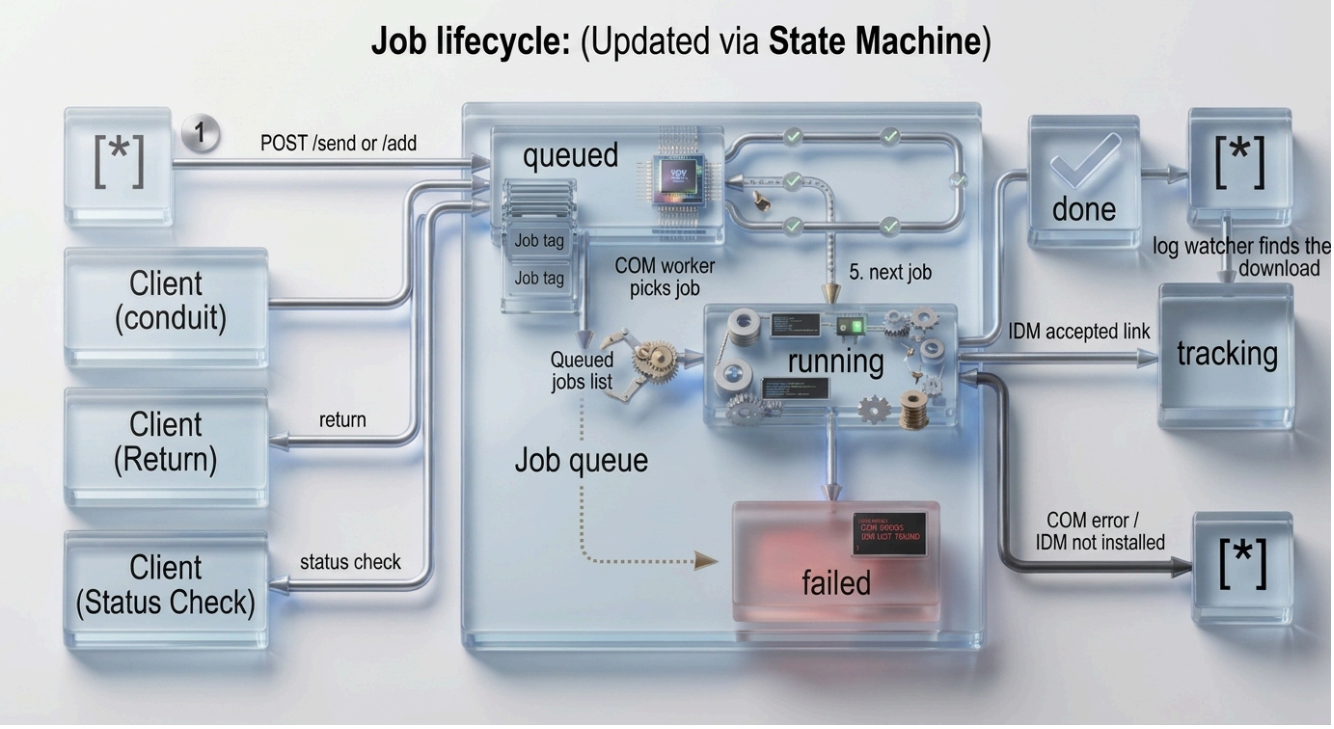
```
{"id": "a1a8d016346c", "url": "http://test.com/test.exe", "status": "done", "state": "downloadin
 "download": {"state": "downloading", "speed": 3000, "downloaded": 15000, "saveas": "C:\\DL\\tes
```


Status	Meaning
`202`	accepted and queued
`400`	invalid JSON, invalid url/field
`401`	missing/wrong token (or cross-site / foreign Host request without token)
`404`	unknown endpoint or job id
`411` / `413`	no `Content-Length` / body larger than 1 MB
`415`	`Content-Type` is not `application/json`

Job lifecycle



Only the last 200 jobs are kept in memory; history is cleared when the agent stops. `GET /downloads` is rebuilt from IDM's logs, so it survives agent restarts (within the 5 minute start window).



FAQ: what is the queue for?

Is the queue only for scheduling downloads? No. The agent queue only serialises *sending links to IDM*: COM objects are bound to one thread, so every request is queued and the COM worker hands the links to IDM one at a time. It does not delay, order or throttle the downloads themselves. Once IDM has the link it downloads immediately (or waits in IDM's own queue with `add_only`).

You want	Use
download now	`POST /send` (default)
put it in IDM's main queue, start later	`add_only: true` / `POST /add` / `idm -a`
time-based scheduling	IDM's own *Scheduler* (Downloads > Scheduler); add the links with `add_only`, then schedule the queue in IDM

Python API

```
from idm import IDMan, AgentServer, agent_send, LogWatcher

# client (any OS)
res = agent_send("192.168.1.10:8899",
                 ["http://test.com/a.exe", "http://test.com/b.exe"],
                 {"referrer": "http://test.com", "add_only": True},
```

```
                  token="SECRET")
print(res)   # {'queued': 2, 'jobs': [...]}

# track completion locally (Windows)
w = LogWatcher()
w.start()
state = w.wait_final("http://test.com/a.exe", since=0)
print(state["state"])   # complete / error / cancelled

# embed the agent in your own program (blocks until Ctrl+C)
AgentServer(IDMan(), host="127.0.0.1", port=8899, token=None,
            defaults={"path": r"C:\DOWNLOADS"}, watcher=LogWatcher()).serve()
```

`agent_send` raises `idm.AgentError` when the agent is unreachable or replies with an error.

Configuration

Config file is `idm.ini` inside the package (change with `idm --config section:option:value`).

Section	Option	Description
`[download]`	`path`	default save path (also used by the agent server when the request has no `path`)
`[download]`	`confirm`	confirm before download
`[data]`	`user_agent`	default User-Agent
`[debug]`	`verbose`	print "Link sent to IDM successfully."
`[server]`	`host`, `port`, `token`	defaults for `idm --server` (`127.0.0.1`, `8899`, none)
`[server]`	`no_webui`	`1` = disable the web interface (same as `--no-webui`)
`[agent]`	`url`, `token`	when `url` is set the CLI always sends to the agent; use `--local` to force local IDM
`[watcher]`	`log_dir`	IDM log directory (default `%APPDATA%\IDM\DwnlData\<USERNAME>`)
`[watcher]`	`disable`	`1` = no completion tracking (same as `--no-watcher`)

Precedence: command line option, then environment `PYIDM_TOKEN` (token only), then `idm.ini`, then built-in default.

Example:

```
> idm --config server:port:9000
> idm --config agent:url:http://192.168.1.10:9000
> idm --config agent:token:SECRET
```

Security

- The agent listens on `127.0.0.1` by default. Binding to any other host is refused unless a token is set.
- Without a token (loopback only) requests with a non-loopback `Host` header are rejected (DNS rebinding), and a request carrying an `Origin` header is accepted only when it is the agent's own origin (the built-in web page). A page on any other site cannot trigger downloads. `Content-Type: application/json` is mandatory for POST and no CORS headers are sent.
- With a token every endpoint except `/health` and the static page requires it (constant-time comparison).
- The web page is served with a strict Content-Security-Policy, `X-Frame-Options: DENY` and renders all data with `textContent` only.
- Traffic is plain HTTP. For access outside a trusted LAN put it behind a VPN or a TLS reverse proxy.
- `path` is a path on the agent machine; anyone holding the token can make the agent save files anywhere IDM can write.
- Credentials sent in `username` / `password` / `cookie` travel in the request body, protect the transport accordingly.

Run at Windows startup

```
schtasks /Create /SC ONLOGON /TN "pyidm-agent" /TR "idm --server --host 0.0.0.0 --token SECRET" /
```

Remove with `schtasks /Delete /TN "pyidm-agent" /F`. The agent must run in the same logged-in user session as IDM (the IDM log directory is per user).

Troubleshooting

Problem	Fix
`Local IDM (COM) is only available on Windows`	you are not on Windows: use `idm -A http://HOST:8899 URL`

Problem	Fix
`COM support is not installed`	`pip install comtypes pywin32` (agent-only install on Windows)
`Cannot reach agent server`	agent not running, wrong host/port, or firewall (allow the port in Windows Firewall)
`HTTP 401: unauthorized`	token mismatch; pass `--agent-token` / `PYIDM_TOKEN`, or the request came from another site / Host
`Refusing to listen on ... without a token`	add `--token` when using a non-loopback `--host`
`HTTP 415`	add `Content-Type: application/json`
job `failed` with COM error	IDM not installed/running in the same session, check `idm --server -v` output
`state` stays `sent`	log watcher off (`"watcher": false` in `/health`): check `[watcher] log_dir` / `--log-dir`, run the agent as the same Windows user as IDM
`Address already in use`	another process uses the port, choose `--port`

Support

- Python 3.7+
- Agent server / local IDM: Windows only (IDM, `comtypes`, `pywin32`)
- Agent client (`idm -A`, `agent_send`, web interface): any OS; for plain downloading on Linux you can also use `pywget` (pip install pywget) or wgetpanel

Tips

`always provide 'referrer', some sites must have referrer in header`

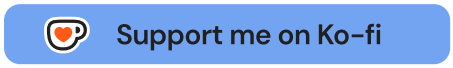
Links

- License: GPL
- Code: https://github.com/cumulus13/pyidm
- Issue tracker: https://github.com/cumulus13/pyidm/issues

Author

Hadi Cahyadi





Support me on Patreon